

# HOUSE PRICE PREDICTION

## ✓ LOAD DATA

```
#Import Libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error
```

```
df = pd.read_csv('/content/House_Price_Prediction_Dataset_a2s++c.csv')
print("Dataset loaded successfully. Displaying the first 5 rows:")
display(df.head())
```

Dataset loaded successfully. Displaying the first 5 rows:

|          | <b>Id</b> | <b>Area</b> | <b>Bedrooms</b> | <b>Bathrooms</b> | <b>Floors</b> | <b>YearBuilt</b> | <b>Location</b> | <b>Condition</b> |
|----------|-----------|-------------|-----------------|------------------|---------------|------------------|-----------------|------------------|
| <b>0</b> | 1         | 1360        | 5               | 4                | 3             | 1970             | Downtown        | Excellent        |
| <b>1</b> | 2         | 4272        | 5               | 4                | 3             | 1958             | Downtown        | Excellent        |
| <b>2</b> | 3         | 3592        | 2               | 2                | 3             | 1938             | Downtown        | Good             |
| <b>3</b> | 4         | 966         | 4               | 2                | 2             | 1902             | Suburban        | Fair             |
| <b>4</b> | 5         | 4926        | 1               | 4                | 2             | 1975             | Downtown        | Fair             |

## ✓ 1. Dataset Information

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   Id          2000 non-null   int64  
 1   Area        2000 non-null   int64  
 2   Bedrooms    2000 non-null   int64  
 3   Bathrooms   2000 non-null   int64  
 4   Floors      2000 non-null   int64  
 5   YearBuilt   2000 non-null   int64  
 6   Location    2000 non-null   object  
 7   Condition   2000 non-null   object  
 8   Garage      2000 non-null   object  
 9   Price       2000 non-null   int64  
dtypes: int64(7), object(3)
memory usage: 156.4+ KB
None
```

## ✓ 2. Descriptive Statistics

```
# Display descriptive statistics for numerical columns
display(df.describe())
```

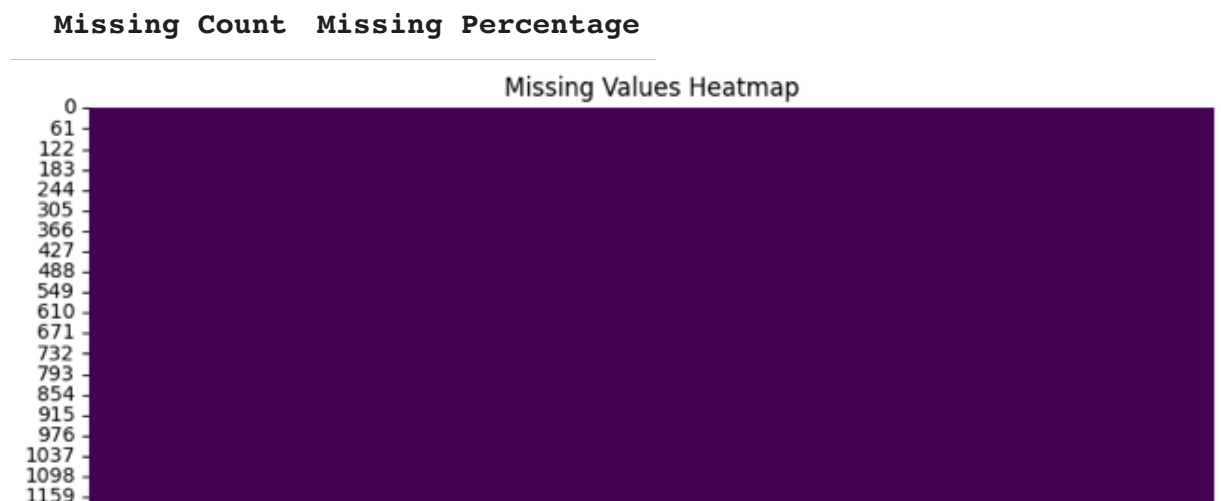
|       | Id          | Area        | Bedrooms    | Bathrooms   | Floors      | Year        |
|-------|-------------|-------------|-------------|-------------|-------------|-------------|
| count | 2000.000000 | 2000.000000 | 2000.000000 | 2000.000000 | 2000.000000 | 2000.000000 |
| mean  | 1000.500000 | 2786.209500 | 3.003500    | 2.552500    | 1.993500    | 1961.400000 |
| std   | 577.494589  | 1295.146799 | 1.424606    | 1.108990    | 0.809188    | 35.900000   |
| min   | 1.000000    | 501.000000  | 1.000000    | 1.000000    | 1.000000    | 1900.000000 |
| 25%   | 500.750000  | 1653.000000 | 2.000000    | 2.000000    | 1.000000    | 1930.000000 |
| 50%   | 1000.500000 | 2833.000000 | 3.000000    | 3.000000    | 2.000000    | 1961.000000 |
| 75%   | 1500.250000 | 3887.500000 | 4.000000    | 4.000000    | 3.000000    | 1993.000000 |
| max   | 2000.000000 | 4999.000000 | 5.000000    | 4.000000    | 3.000000    | 2023.000000 |

## ✓ 3. Missing Values Analysis

```
# Calculate and display the percentage of missing values for each c
missing_values = df.isnull().sum()
missing_percentage = (missing_values / len(df)) * 100
missing_df = pd.DataFrame({'Missing Count': missing_values, 'Missin

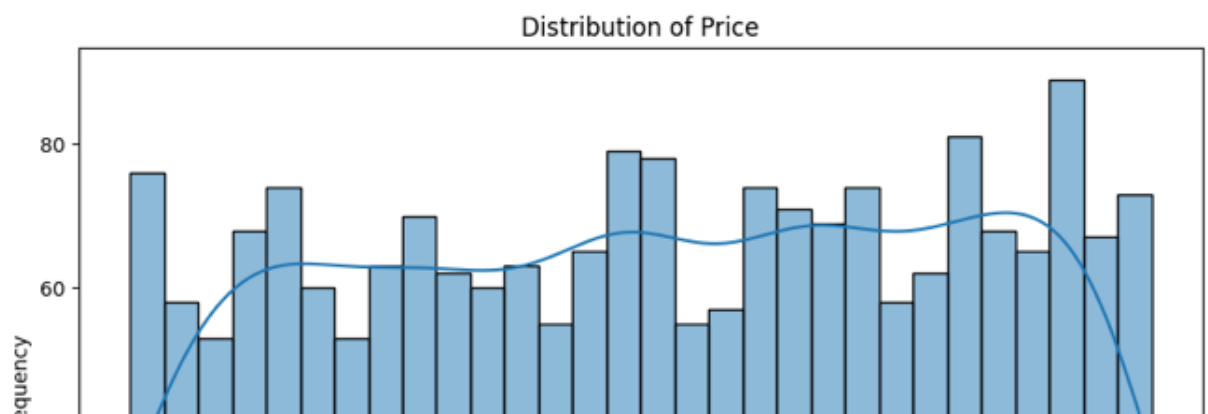
# Filter to show only columns with missing values and sort them
display(missing_df[missing_df['Missing Count'] > 0].sort_values(by=

# Visualize missing values (optional, for larger datasets this help
plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap')
plt.show()
```



#### ✓ 4. Distribution of the Target Variable (SalePrice)

```
# Plot the distribution of 'SalePrice' using a histogram and a KDE
plt.figure(figsize=(10, 6))
sns.histplot(df['Price'], kde=True, bins=30)
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```

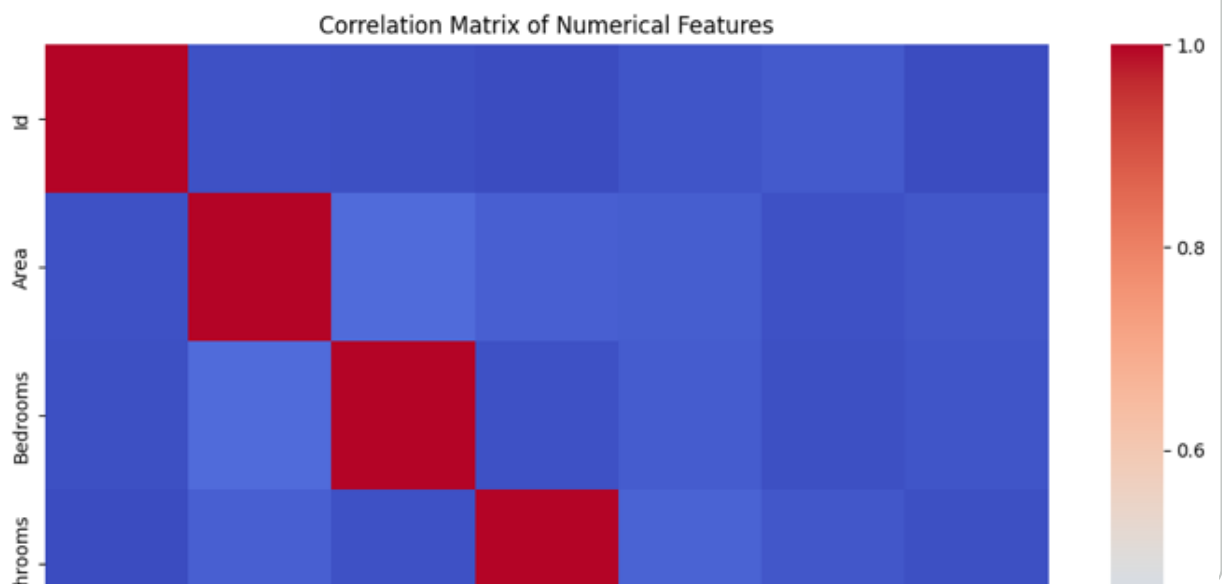


## ✓ 5. Correlation Matrix of Numerical Features

```
# Select only numerical columns for correlation calculation
numerical_df = df.select_dtypes(include=np.number)

# Calculate the correlation matrix
correlation_matrix = numerical_df.corr()

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=False, cmap='coolwarm', fmt="")
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



## ✓ 6. Analyze Categorical Features

```
Identify categorical columns
categorical_cols = df.select_dtypes(include='object').columns

print(f"Found {len(categorical_cols)} categorical columns: {list(categorical_cols)}")

# For each categorical column, display unique values and value counts
for col in categorical_cols:
    print(f"\n--- Column: {col} ---")
    print(df[col].value_counts())

# Plot distribution against Price if there are not too many unique values
if df[col].nunique() < 20:
    plt.figure(figsize=(12, 6))
    sns.boxplot(x=col, y='Price', data=df)
    plt.title(f'Price Distribution by {col}')
    plt.xlabel(col)
    plt.ylabel('Price')
```

```
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Found 3 categorical columns: ['Location', 'Condition', 'Garage']

--- Column: Location ---

Location

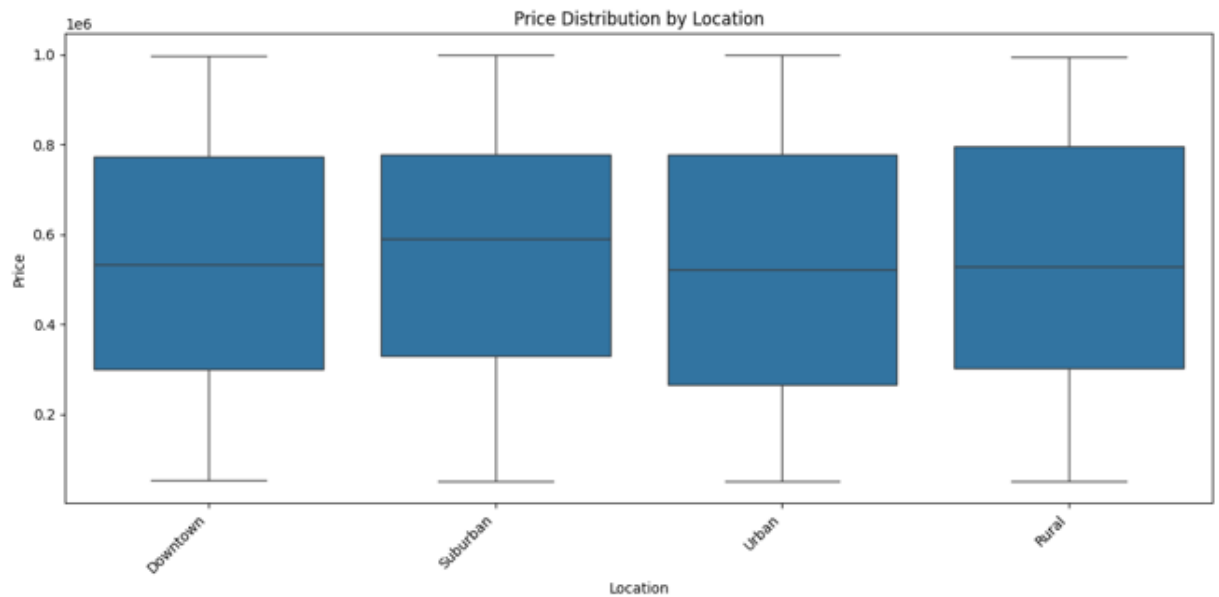
Downtown 558

Urban 485

Suburban 483

Rural 474

Name: count, dtype: int64



--- Column: Condition ---

Condition

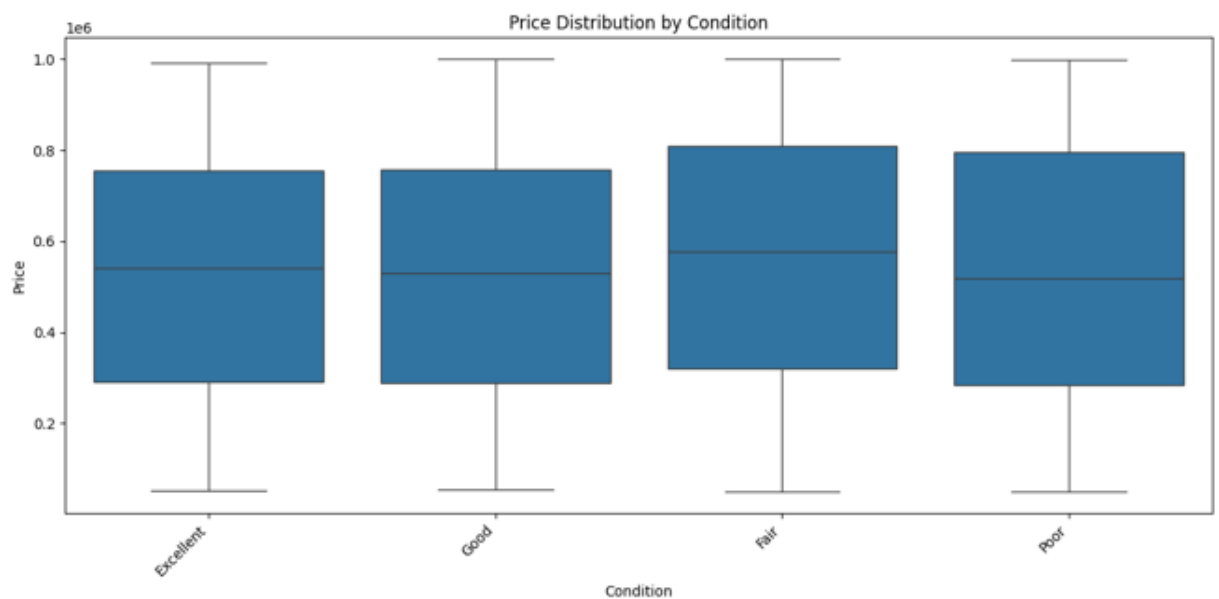
Fair 521

Excellent 511

Poor 507

Good 461

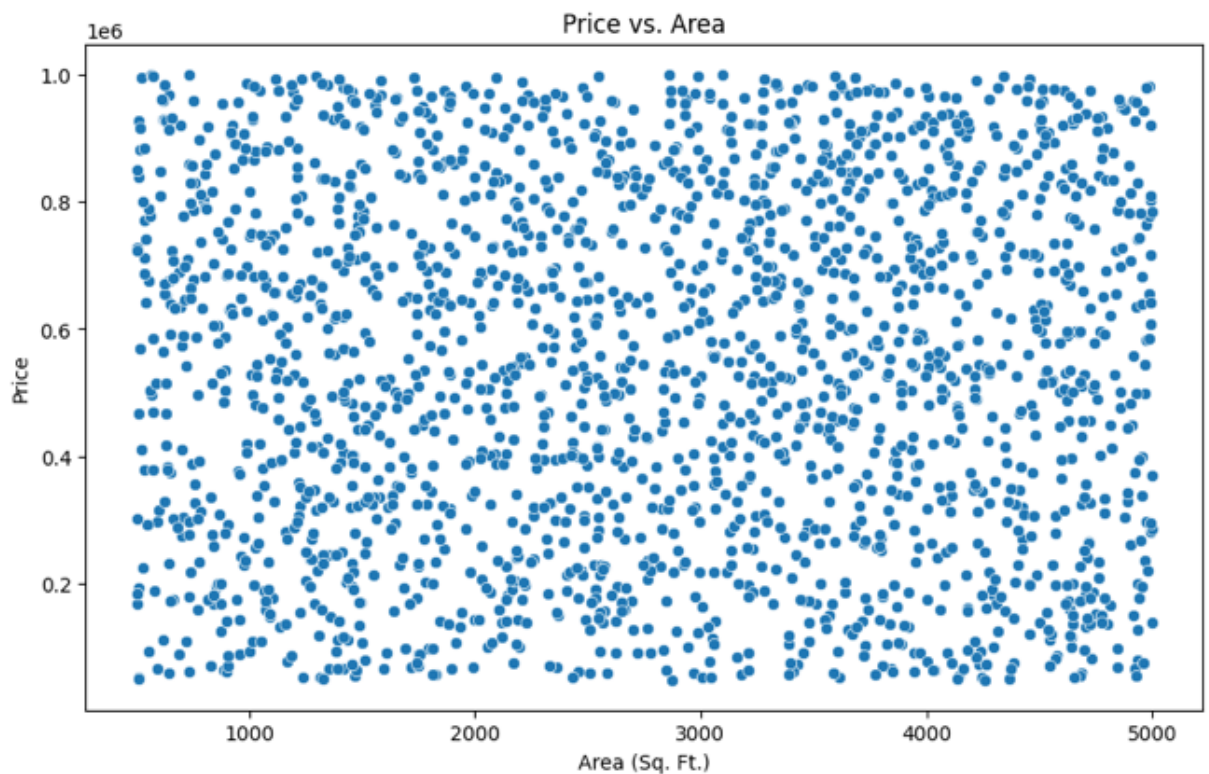
Name: count, dtype: int64



## ✓ 7. Visualize Key Numerical Features vs. Price

```
# Example: Relationship between 'Area' (Above grade (ground) living
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Area', y='Price', data=df)
plt.title('Price vs. Area')
plt.xlabel('Area (Sq. Ft.)')
plt.ylabel('Price')
plt.show()
```

```
# Example: Relationship between 'Bedrooms' and 'Price'
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Bedrooms', y='Price', data=df)
plt.title('Price vs. Bedrooms')
plt.xlabel('Bedrooms (no.)')
plt.ylabel('Price')
plt.show()
```



## ✓ 8. Outlier Detection in Key Numerical Features

```
# Box plot for 'Price' to visualize outliers
plt.figure(figsize=(10, 6))
sns.boxplot(y=df['Price'])
plt.title('Box Plot of Price (Outlier Detection)')
plt.ylabel('Price')
plt.show()
```

